

一、简答题

1.(1)算法分析是指对一个算法的运行时间和占用空间做定量的分析,计算相应的数量级,并用时间复杂度和空间复杂度表示。分析算法的目的就是要降低算法的时间复杂度和空间复杂度,提高算法的执行效率。

(2)输入、输出、有穷性、可行性、确定性

(3)当问题规模 n 趋于无穷大时,影响算法效率的重要因素是 $T(n)$ 的数量级,其他低阶项主要受高阶项所制约,因此可以用 $T(n)$ 的数量级评价算法。时间复杂度 $T(n)$ 的数量级称为渐进时间复杂性。

2.

2. 1) 仍是线性的.

$\therefore$ 划分为 7 个一组

$\therefore$ 每次能排除的元素个数至少为: $4 \left(\left\lfloor \frac{n}{7} \right\rfloor + 1 \right) \geq \frac{2n}{7}$

$\therefore$ 剩余搜索规模为 $n - \frac{2n}{7} = \frac{5n}{7}$

$\therefore$ 递归方程为 $T(n) \leq T(\frac{n}{7}) + T(\frac{5n}{7}) + cn$

可利用递归树求解该方程.

该树的深度为 $\log_{\frac{7}{5}} n$

$\therefore T(n) = \sum_{i=0}^{\infty} \left(\frac{6}{7}\right)^i cn$

$\lim_{n \rightarrow \infty} T(n) = 7cn$

$\therefore T(n) = O(n).$

2) $\therefore$ 划分为 3 个一组.

$\therefore$ 每次排除元素个数至少为: $2 \left(\left\lfloor \frac{n}{3} \right\rfloor + 1 \right) \geq \frac{n}{3}$

$\therefore$ 递归方程为 $T(n) \leq T(\frac{n}{3}) + T(\frac{2n}{3}) + cn.$

$\therefore T(n) = cn \cdot \log_{\frac{3}{2}} n = O(n \lg n).$

$\therefore$ 划分为 3 个一组运行时间不是线性的.

3. 采用公式法.

$$\therefore f(n) = 4f(n/2) + n$$

$$\therefore a = 4, b = 2, f(n) = n$$

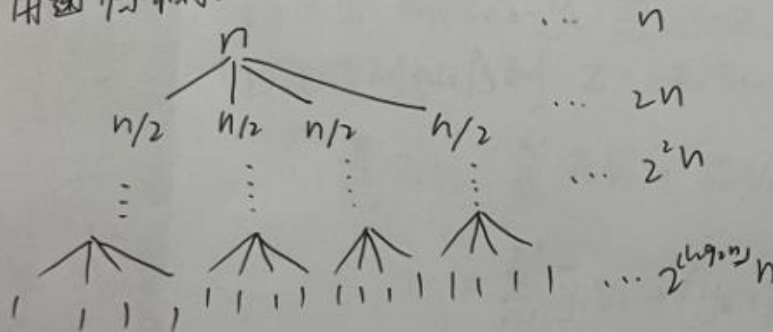
$$\therefore \log_b a = 2 \quad n^{\log_b a} = n^2$$

$$\therefore \exists \varepsilon = 1, \varepsilon > 0, f(n) = O(n^{\log_b a - \varepsilon}) = O(n^{2-1}) = O(n)$$

$$\therefore f(n) = \Theta(n^2)$$

解法二.

用递归树.



$$\therefore f(n) = n + 2n + \dots + 2^{\lg n} n$$

$$= (1 + 2 + 2^2 + \dots + 2^{\lg n}) n$$

$$\leq (2^{\lg n + 1} - 1) n$$

$$= \cancel{n} 2n^2 - n$$

4. 假设一共有 N 件物品, 第 i 件物品的价值为 v_i , 重量为 w_i , 背包能容纳的最大重量为 W
算法:

1. 将物品按单位重量所具有的价值排序, 即按照 v_i/w_i 降序排列, 得到新的物品序列
2. 优先选择当前单位重量下价值最大的物品, 若将这种物品全部装入背包后, 背包还有多余容量, 则选择单位重量价值次高的并尽可能多地装入背包。如果最后一件物品无法全部装入, 则计算可以装入的比例, 然后按比例装入。

证明:

假设有最优解 $X = (x_1, x_2, \dots, x_n)$ 且由算法可知存在 j , 对于所有 $a > j$, $x_a = 0$ 成立.

即 $X = (x_1, x_2, \dots, x_j, 0, \dots, 0)$ 有最优解 $Y = (y_1, y_2, \dots, y_n)$.

设 k 为序列 X 和 Y 的第一对不相等的数的下标, 即对于所有 $a \in [1, k)$, $x_a = y_a$ 成立.

① 若 k 不存在, 则 $X = Y$, $\therefore X$ 为最优解.

② 若 k 存在, 易证 $k \leq j$. 假设 $k > j$, 则 $\sum_{i=1}^k y_i w_i > \sum_{i=1}^j y_i w_i = \sum_{i=1}^j x_i w_i = W$ (矛盾).

现调整 $y_a, a \in [k+1, n]$, 使得 y_a 减去一定的数量并加到 y_k 上, 且始终保持不变, 由此得到新的序列 $Z = (z_1, z_2, \dots, z_n)$ $z_k w_k - y_k w_k = \sum_{i=k+1}^n (y_i - z_i) w_i$

$$\therefore \sum_{i=1}^n z_i v_i = \sum_{i=1}^k y_i v_i + y_k v_k + v_k (z_k - y_k) + \sum_{i=k+1}^n y_i v_i - \sum_{i=k+1}^n y_i v_i - z_i v_i$$

$$= \sum_{i=1}^k y_i v_i + v_k (z_k - y_k) - \sum_{i=k+1}^n (y_i - z_i) v_i$$

$$= \sum_{i=1}^k y_i v_i + (z_k - y_k) \cdot \frac{w_k}{v_k} \cdot \frac{v_k}{w_k} - \sum_{i=k+1}^n (y_i - z_i) \cdot w_i \cdot \frac{v_i}{w_i}$$

$$\therefore (z_k - y_k) w_k = \sum_{i=k+1}^n (y_i - z_i) w_i$$

$$\text{又} \because \frac{v_k}{w_k} \geq \frac{v_i}{w_i}, i \in [k+1, n]$$

$$\therefore (z_k - y_k) w_k \cdot \frac{v_k}{w_k} \geq \sum_{i=k+1}^n (y_i - z_i) w_i \cdot \frac{v_i}{w_i}$$

$$\therefore \sum_{i=1}^n z_i v_i \geq \sum_{i=1}^n y_i v_i$$

$\therefore Z$ 也是最优解.

那么循环修改 X 与 Y 中不同的值, 可证明 X 也是最优解.

二、

1、

写法很多, 有填数组的、画图的, 重点是体现求解过程, 让人理解算法流程
打分以助教能否理解你的算法为标准

扣分情况:

没写动态规划数组表示的意义的适当扣分

用 Dijkstra 的适当扣分

只写伪代码的适当扣分

参考答案:

从前往后 (1→8)

DP[i] 表示 1 到点 i 的最短距离

	1	2	3	4	5	6	7	8
第一步	0							
第二步	0	3	5	2				
第三步	0	3	5	2	4	6	7	
第四步	0	3	5	2	4	6	7	8

需要写一下状态更新公式如 $dp[2]=C(1,2)=3$, $dp[7]=\min(dp[2]+C(2,7))$

多段图划分为 4 个点集 V_1, V_2, V_3, V_4

$D(i,j)$ 表示点集 i 中的点 j 到源点 1 的距离

$$D(1,1)=0$$

$$D(2,2)=C(1,2)=3$$

$$D(2,3)=C(1,3)=5$$

$$D(2,4)=C(1,4)=2$$

$$D(3,5)=\min(C(3,5)+D(2,3), C(4,5)+D(2,4))=4$$

$$D(3,6)=\min(C(2,6)+D(2,2), C(3,6)+D(2,3), C(4,6)+D(2,4))=3$$

$$D(3,7)=\min(C(2,7)+D(2,2))=7$$

$$D(4,8)=\min(C(5,8)+D(3,5), C(6,8)+D(3,6), C(7,8)+D(3,7))=8$$

(从后往前 $8 \rightarrow 1$)

$D(i,j)$ 表示点集 i 中的点 j 到目标点 8 的最短距离

$$D(4,8)=0$$

$$D(3,5)=C(5,8)=4$$

$$D(3,6)=C(6,8)=5$$

$$D(3,7)=C(7,8)=6$$

$$D(2,2)=\min(C(2,6)+D(3,6), C(2,7)+D(3,7))=10$$

$$D(2,3)=\min(C(3,5)+D(3,5), C(3,6)+D(3,6))=9$$

$$D(2,4)=\min(C(4,5)+D(3,5), C(4,6)+D(3,6))=6$$

$$D(1,1)=\min(C(1,2)+D(2,2), C(1,3)+D(2,3), C(1,4)+D(2,4))=8$$

$$1 \rightarrow 4 \rightarrow 5 \rightarrow 8$$

$$1 \rightarrow 4 \rightarrow 6 \rightarrow 8$$

三、

1、

由 $A(n,m)=A(A(n-1,m),m-1)$ 公式可知, 计算 $A(n,m)$ 会用到 $A(x,m-1)$

按照动态规划的思想，我们可以用数组缓存 $A(x, m-1)$ 的结果，但由于 $x=A(n-1, m)$ 的值不确定，难以确定数组的大小（实际上阿克曼函数 $m=3$ 后就会变得很大）。

我们换一个思路，当我们计算 $A(n, m)$ 时，我们会用到 $m-1$ 层中的某个值，也就是说 $A(i, m)$ 的值全部都能在第 $m-1$ 层找到。同理 $m-1$ 层的值全部都能在第 $m-2$ 层找到。以此类推，第 m 层的值全部都能在第 0 层找到。

思路：计算 $m=0$ 时阿克曼函数的值，每计算一个 $A(i, 0)$ ，我们找到 $m=1$ 层中对应的位置 $A(j, 1)$ ，

此时， $A(j, 1)=A(A(j-1, 1), 0) \rightarrow i=A(j-1, 1)$ 。

i 为上一层计算到的那一列， $A(j-1, 1)$ 为下一层某个值

使用两个数组 ind, val 分别保存第 $m=i$ 层计算到的列，和第 i 层计算到的值
每计算一个 $A(i, 0)$ ，更新 $ind[0-m]$ 和 $val[0-m]$

$A(1, m)=A(A(0, m), m-1)=A(1, m-1)=\dots=A(1, 0)=2$

$A(2, m)=A(A(1, m), m-1)=A(2, m-1)=\dots=A(2, 0)=2+2=4$

从 $n=3$ 开始算

```
int ack(int m, int n){
    int i;
    int* ind; //ind 记录当前第 i 行计算到第 ind[i] 列的值
    int* val; //val[i]=A(i, ind[i])
    if(n==0) return 1;
    if(n==1 && m==0) return 2;
    if(n==1) return 2;
    if(n==2) return 4;
    val=new int[m+1];
    ind=new int[m+1];
    for(i=0; i<=m; i++){
        val[i]=4;
        ind[i]=2;
    }
    val[0]=4; //从第 0 行开始
    ind[0]=2; //第 0 行第 ind[0] 列 (3)
    while(ind[m]<n){ //一直计算，知道第 m 层第 n 列计算好后返回
        val[0]++; //更新第 0 层的值，A(n, 0)=n+2
        ind[0]++; //更新第 0 层的列
        for(i=0; i<m; i++){
            //每更新一个第 0 层的值，往下找对应的位置
            if(val[i+1]!=ind[i])
                break;
            ind[i+1]++;
            val[i+1]=val[i];
        }
    }
}
```

```

    }
    return val[m];
}

```

Ack(m,n)矩阵: m 行, n 列, 从 0 开始

```

1 2 4 5 6 7 8 9 10 11 12
1 2 4 6 8 10 12 14 16 18 20
1 2 4 8 16 32 64 128 256 512 1024
1 2 4 16 65536

```

2、LeetCode312

暴力枚举:

枚举戳气球的顺序 (相当于全排列)

时间复杂度 $O(n!)$

空间复杂度 $O(n)$

动态规划:

我们观察戳气球的操作, 发现这会导致两个气球从不相邻变成相邻, 使得后续操作难以处理。于是我们倒过来看这些操作, 将全过程看作是每次添加一个气球。

令 $dp[i][j]$ 表示填满开区间 (i,j) 能得到的最多硬币数, 那么边界条件是 $i \geq j-1$, 此时有 $dp[i][j]=0$ 。

可以写出状态转移方程:

$$dp[i][j] = \begin{cases} \max_{k=i+1}^{j-1} val[i] \times val[k] \times val[j] + dp[i][k] + dp[k][j], & i < j-1 \\ 0, & i \geq j-1 \end{cases}$$

```

int maxCoins(vector<int>& nums) {
    int n = nums.size();
    vector<vector<int>> rec(n + 2, vector<int>(n + 2));
    vector<int> val(n + 2);
    val[0] = val[n + 1] = 1;
    for (int i = 1; i <= n; i++) {
        val[i] = nums[i - 1];
    }
    for (int i = n - 1; i >= 0; i--) {
        for (int j = i + 2; j <= n + 1; j++) {
            for (int k = i + 1; k < j; k++) {

```

```
        int sum = val[i] * val[k] * val[j];
        sum += rec[i][k] + rec[k][j];
        rec[i][j] = max(rec[i][j], sum);
    }
}
return rec[0][n + 1];
}
```

时间复杂度 $O(n^3)$ 空间复杂度 $O(n^2)$